

Proof under Total Correctness 2: Proof of Convergence

- Remember that we only create proofs for provable triples. If a statement contains a loop that can be proved to be convergent, then this loop must iterate for a finite number of iterations.
 - For example, let $S \equiv k := 0; \text{while } k < 5 \text{ do } k := k + 1 \text{ od}$, then while loop iterates 5 iterations. When k gets increased to 2, including the current iteration, the loop still has 3 iterations.
- In general, we cannot always find the exact number of (remaining) iterations a loop has, but if we can upper bound the number of (remaining) iterations of a loop by a finite natural number and the number decreases after each iteration, then we can prove the loop converges.
 - For example, let $W \equiv \text{while } k < n \text{ do } k := k + 1 \text{ od}$, we don't have an exact number of iterations; but we can use $n - k$ to express the number of (remaining) iterations before W terminates, and $n - k$ is decreasing after each iteration.
- A **bound expression** or **bound function** t for a loop is a finite *natural number* expression that, at each loop test, gives a **strictly decreasing** upper bound on the number of iterations remaining before termination. A bound expression can use program variables (variables in statement) and logical variables (variables in precondition or postcondition).
- We'll attach the bound expression t to a loop using the syntax **{bd t }**, thus in a proof outline for total correctness, a loop usually looks like: **{inv p }{bd t } while B do S od**.

(Properties of Bound Expressions)

What properties do we need for t so that it can be bound for loop $W \equiv \{\text{inv } p\} \{\text{bd } t\} \text{while } B \text{ do } S \text{ od}$? Here we list two requirements.

- $p \Rightarrow t \geq 0$.
 - Loop invariant p is always true after each iteration of a while loop, it needs to logically imply that there are non-negative iterations left to do.
- $\vdash_{tot} \{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$
 - t_0 is the value of t before the execution of one iteration; after the execution of this iteration, the value of t needs to be strictly decreased to guarantee convergence.

- Show a partial proof outline under total correctness to show that the loop in the following program converges and p is a correct loop invariant.

```

{ $n \geq 0$ }
 $k := 0; s := 0;$ 
{inv  $p \equiv 0 \leq k \leq n \wedge s = \text{sum}(0, k)$ }
{bd ?} while  $k < n$  do
     $s := s + k + 1; k := k + 1$ 
od
{ $s = \text{sum}(0, n)$ }
  
```

- First, let's look for a bound function, it needs to be non-negative, and it needs to be reduced after each iteration. $n - k$ is an easy choice: loop invariant implies that $n \geq k$ and k is increased by 1 after each iteration.

- We can get the following partial proof outline including only conditions about the loop.

```

{n ≥ 0}
k := 0; s := 0;
{inv p ≡ 0 ≤ k ≤ n ∧ s = sum(0, k)}
{bd n - k} while k < n do
  {p ∧ k < n ∧ n - k = t0}
  {p[k + 1 / k][s + k + 1 / s] ∧ n - (k + 1) < t0}
  s := s + k + 1; {p[k + 1 / k] ∧ n - (k + 1) < t0}
  k := k + 1
  {p ∧ n - k < t0}
od
{p ∧ k ≥ n}
{s = sum(0, n)}

```

2. Let $W \equiv \{\text{inv } p\} \{\text{bd } t\} \text{ while } B \text{ do } S \text{ od}$. Decide true or false for each of the following statements about loop bound expressions.
 - a. t can be a constant.
False. A constant cannot be reduced after each iteration. So, in the example **while** $k < 5$ **do** $k := k + 1$ **od**,
 - b. $t \geq 0 \Rightarrow B$.
False. Since $p \Rightarrow t \geq 0$, so if $t \geq 0 \Rightarrow B$ then $p \Rightarrow B$ after each iteration, which means the loop diverges and it is a contradiction to the existence of t .
 - c. $p \wedge B \Rightarrow t > 0$.
True. If $p \wedge B$, then we will enter the loop body, which means there is at least another iteration; thus t is strictly greater than 0 since its value needs to be reduced by at least 1 after this iteration.
 - d. $p \wedge t = 0 \Rightarrow \neg B$.
True. This is the contra-positive of c.
 - e. $\models_{tot} \{p \wedge B \wedge t = t_0\} S \{t = t_0 - 1\}$
False. We don't require t reduce by exactly 1 after each iteration.
 - f. Let N be an expression showing the exact number of remaining iterations of W , then N is $\Theta(t)$.
False. We don't require t is a tight upper bound of N .
 - g. $p \wedge \neg B \Rightarrow t = 0$.
False. We don't require t to go down to 0 after the last iteration.
- There is no algorithm to find a bound expression for a loop. But here we have some heuristics based on the requirements for bound expression:
 - a) We start with $t \equiv 0$.
 - b) In the loop body, if the value of a variable x gets decreased then we CAN concatenate $+x$ to t ; if the value of a variable y gets increased then we CAN concatenate $-y$ to t .

- c) If p does not logically imply $t \geq 0$, adjust the constant for each term or add constant terms to the expression.

3. Find a bound expression for the following loops:

- a. **{inv $n \leq x + y$ }{bd ?} while $x + y > n$ do $y := y - 1$ od**
 o Following the first two steps of the above guidelines, we can try to use $0 + y$ as a bound expression. It is strictly decreased after each iteration but there is no evidence that $y \geq 0$. From loop invariant we know that $n \leq x + y$, so $x + y - n \geq 0$; x and n doesn't change after each iteration, so $x + y - n$ strictly decrease after each iteration. So, $x + y - n$ is a bound expression.
- b. **{inv $n \leq 2 * y - x$ }{bd ?} while $y > n + x$ do $y := y - 1$ od**
 o Similarly, we can try to use y as a bound expression but there is no evidence that $y \geq 0$. From loop invariant we know that $n \leq 2 * y - x$, so $2 * y - x - n \geq 0$; x and n doesn't change after each iteration, so and $2 * y - x - n$ is a bound expression.
- c. **{inv $x + y < n$ }{bd ?} while $x + y < n$ do $y := y - 1; x := x + 2$ od**
 o In each iteration, x is increased and y is decreased; we can try to use $-x + y$ as a bound expression but there is no evidence showing that $-x + y \geq 0$. Even if we adjust the constant, we still cannot find an expression with $-x$ and $+y$ that's both non-negative and decreasing. Then we noticed that $(x + y)$ as whole increases after each iteration, and the loop invariant tell us $n - (x + y) > 0$; so $n - (x + y)$ is a bound expression.

$$\begin{array}{l} n > x + y \\ n - (x + y) > 0 \end{array}$$

Loop Rule under Total Correctness

- *In our assignments and exams, there won't be questions that need the total correctness version of the While Loop Rule: possible runtime errors and possible divergence won't appear in the same proof under total correctness.
- To show that a loop $W \equiv \{\text{inv } p\}\{\text{bd } t\} \text{ while } B \text{ do } S \text{ od}$ is totally correct, we need:
 - p is a loop invariant and p is safe.
 - No runtime error while evaluating B or executing S : $p \Rightarrow D(B)$ and $p \wedge B \Rightarrow D(S)$
 - p can logically imply the bound expression t being at least 0 and safe.
 - Loop bound t is decreased after each iteration: $\vdash_{tot} \{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$.

Thus, we have

***While Loop Rule** (total correctness version):

- $p \Rightarrow D(B)$ # Here p is a safe predicate
- $p \wedge B \Rightarrow D(S)$
- $p \Rightarrow \downarrow (t \geq 0)$
- $\{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$
- {inv p }{bd t } while B do S od** $\{p \wedge \neg B\}$ loop 1,2,3,4

Since "possible runtime errors and possible divergence won't appear in the same proof under total correctness" in our assignments and exams, so we assume that $D(B) \equiv T$, $D(S) \equiv T$ and $D(t) \equiv T$, so the above rule will be simplified to the following version:

While Loop Rule (total correctness version for no possible runtime errors):

- $p \Rightarrow t \geq 0$
- $\{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$

3. $\{\text{inv } p\}\{\text{bd } t\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}$ loop 1,2

You will find line 1 and 2 are exactly the two requirements we need for bound expressions.

Finding Loop Invariants and Creating Loops

- It is not always easy to create a loop that works correctly, and finding a good loop invariant is usually the most important part while creating a loop. Usually, after finding the loop invariant, the rest of the loop comes naturally.
- Like loop bound expressions, there is no algorithm to find a loop invariant; but there are some heuristics, and they don't work in all cases.
- While introducing loop invariants, we discussed some basic needs of a loop invariant. Let loop $W \equiv \{\text{inv } p\} \text{ while } B \text{ do } S \text{ od}$ and if we need to show $\{p_0\} W \{q\}$, we need:
 - $p_0 \Rightarrow p$
 - $\{p \wedge B\} S \{p\}$ is provable.
 - $p \wedge \neg B \Rightarrow q$

When we create a loop in a program, we don't have the loop body S and we don't have the loop condition B yet, all we have are conditions p_0 and q .

- To get a loop invariant, we **usually start with q and try to weaken it**. Here are the reasons:
 - 1) q is the postcondition here, it is usually an expectation from the user, so it is usually given to us.
 - 2) We observe that $p \wedge \neg B \Rightarrow q$, but p itself should not be stronger than q : or else, there is no need to have W and the postcondition is already satisfied.
 - 3) While weakening the postcondition q , we usually can get some insight on B at the same time.
- Here are some tricks we usually use to weaken a predicate:
 - 1) Replacing a constant or an expression with a variable: for example, we can weaken $2 + x = z$ to $y + x = z$ where y is a variable (whose domain includes 2).
 - 2) Adding a disjunct: for example, we can weaken $x = 2$ to $x \leq 2$ (because $x \leq 2 \Leftrightarrow x < 2 \vee x = 2$).
 - 3) Removing a conjunct: for example, we can weaken $1 < x \leq 3$ to $x \leq 3$ (because $1 < x \leq 3 \Leftrightarrow 1 < x \wedge x \leq 3$).
- Using these tricks won't guarantee a loop invariant, they can only find some candidate for loop invariants. We still need to try to create loops with these candidates to see whether they make sense.